PipeDoctor Step 2, decoded

What your first Docker run really did, in 7 steps, with the files and lines of code behind each one.

Sprint 1 · Step 2 (local lakehouse) · run of Thursday 8 October 2026
Read it next to `docs/guide/02-code-walkthrough.md` and keep VS Code open on the code.

Your run is green. Raw taxi files went in; three Delta tables came out; the second run changed nothing.

- **It works:** 8 of 8 tests pass. 9 554 778 raw rows → 9 417 635 clean trips + 137 142 quarantined → 20 623 gold rows.
- **It is safe to re-run** (idempotent): run 2 skipped the 3 files already loaded, added 0 rows to silver, and printed the same counts.

This is exactly the healthy baseline PipeDoctor needs: a pipeline that works, that we understand, and that we can break on purpose later.

1 The whole picture on one page

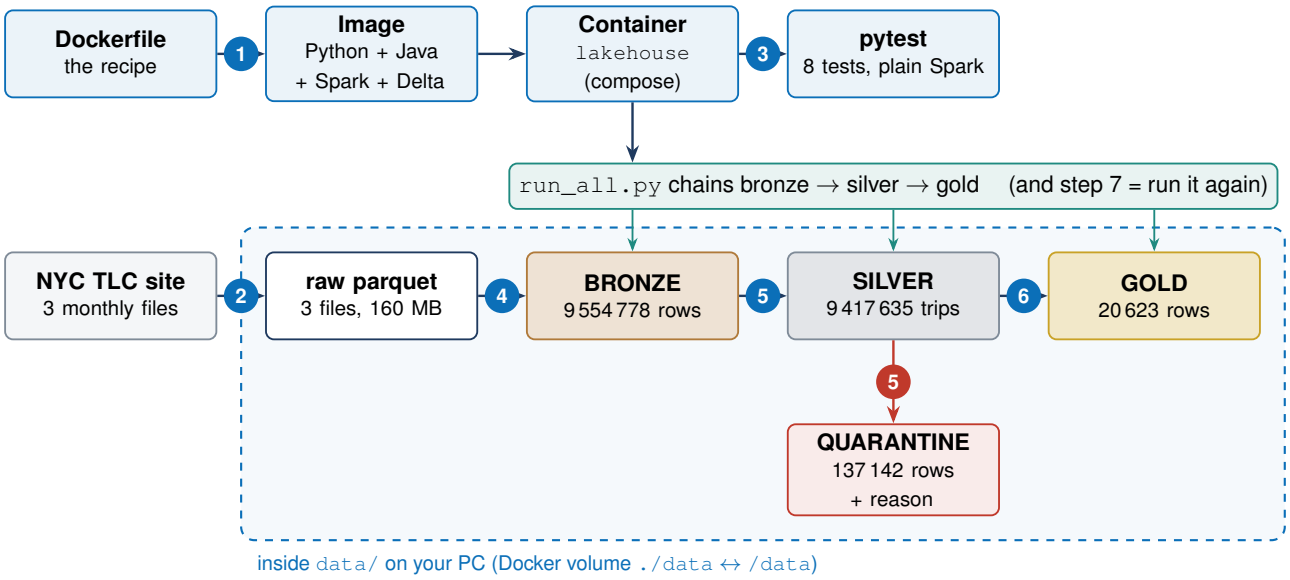


Figure 1. The seven numbered steps. Blue = tooling (steps 1 and 3), grey = outside world, coloured = the data layers. Step 7 is step 4–6 run a second time.

Six words you will hear all project long

Word	In plain words	Where it lives
Image	A frozen computer set-up (Python, Java, Spark, Delta) built once from a recipe.	`docker/Dockerfile`
Container	A running copy of the image. Our commands run *inside* it; with `--rm` it is deleted afterwards.	started by `docker-compose.yml`

Word	In plain words	Where it lives
Volume	A folder shared between your PC and the container, so files survive after the container is gone.	`docker-compose.yml` lines 7–9
Delta table	Parquet files plus a `_delta_log/` folder that records every change. This gives us history, constraints and MERGE.	`data/lakehouse/...`
Medallion	Bronze = raw as received, silver = clean and typed, gold = business view.	`pipelines/*.py`
Idempotent	Run it twice, get the same result. Retries become safe.	`bronze.py` line 33, `silver.py` line 36

2 The 7 steps at a glance

#	Step	Code that does the work	What your output showed
1	**Build the image**	`docker/Dockerfile`, `docker/requirements.txt`	`Image pipedoctor-lakehouse Built 150.2s`
2	**Download the data**	`scripts/download_taxi_data.py`	3 files: 50.0 + 50.3 + 60.1 MB
3	**Unit tests**	`pipelines/tests/test_transforms.py`	`8 passed in 34.78s`
4	**Bronze**	`pipelines/bronze.py`, `transforms.py:61`	`9,554,778 rows in bronze`
5	**Silver**	`pipelines/silver.py`, `transforms.py:73-149`	`9,417,635 trips, 137,142 quarantined`
6	**Gold**	`pipelines/gold.py`, `transforms.py:152`	`20,623 rows, green in 567s`
7	**Re-run (idempotence)**	same code, skip and MERGE logic	same counts, `green in 134s`

Careful with the word "step". The numbers 1–7 in this document are the seven stages of *this run*. The sprint plan has its own steps (Step 3 = GitHub and CI, and so on).

You typed only **five different commands**; steps 4, 5 and 6 are one command (`run_all`), and step 7 is that same command a second time. The extra `pytest` run in your log is a repeat of step 3 and proves nothing new.

3 Step 1 — Build the image

> **Goal.** Build a reusable "box" holding everything the pipeline needs, so it runs identically on your PC, in GitHub CI later, and in PipeDoctor's sandbox.

Command

```
docker compose build
```

`docker-compose.yml:3-5` tells Compose to build from `docker/Dockerfile` using the project folder as context.

Code behind it — and the answer to "is this automated?"

No magic and no hidden script: the **Dockerfile is our own recipe**. Docker runs it line by line, and each instruction becomes one cached *layer* (your log shows `[1/6]` to `[6/6]`). The commands *inside* the layers (`apt-get`, `pip`) are standard tools.

```
4  RUN apt-get update \
5   && apt-get install -y --no-install-recommends openjdk-17-jre-headless procps \
6   && rm -rf /var/lib/apt/lists/*
7
8  COPY docker/requirements.txt /tmp/requirements.txt
9  RUN pip install --no-cache-dir -r /tmp/requirements.txt
```

- `Dockerfile` (line 2): start from the official slim Debian + Python 3.11 image (log `#7`, 18.9 s: layers downloaded).
- `Dockerfile` (lines 4–6): install **Java 17**. Spark runs on the JVM; "PySpark" is only the Python remote control. 27 packages, 53.8 MB (log `#8`, 22.2 s).
- `Dockerfile` (lines 8–9): install the **pinned** versions of `docker/requirements.txt`: `pyspark==3.5.3`, `delta-spark==3.2.1`, `pytest==8.3.3`. Pinned because Delta 3.2 is built for Spark 3.5; mixing versions gives cryptic Java errors (log `#10`, 61.2 s, PySpark alone is 317 MB).

```
11 # Pre-fetch the Delta jars at build time so runs do not need internet for them.
12 RUN python -c "\
13 from delta import configure_spark_with_delta_pip as c; \
14 from pyspark.sql import SparkSession as S; \
15 b = (S.builder.master('local[1]') \
16      .config('spark.sql.extensions','io.delta.sql.DeltaSparkSessionExtension') \
17      .config('spark.sql.catalog.spark_catalog','org.apache.spark.sql.delta.catalog.
       DeltaCatalog')); \
18 s = c(b).getOrCreate(); s.range(1).write.format('delta').save('/tmp/warmup'); s.stop
       ()"
```

- `Dockerfile` (lines 11–18): the **warm-up**. It starts Spark once and writes a 1-row Delta table so the three Delta jars are downloaded *now* (log `#11`: `downloading ... delta-spark_2.12-3.2.1.jar`, 13.9 s). **Proof it worked:** in the later runs the log says `0 artifacts copied, 3 already retrieved`.

```
20 WORKDIR /workspace
21 ENV PYTHONPATH=/workspace \
22     PYTHONUNBUFFERED=1 \
23     LAKEHOUSE_DIR=/data/lakehouse \
24     RAW_DIR=/data/raw
```

- `Dockerfile` (lines 20–24): `PYTHONPATH` lets `import pipelines` work; `LAKEHOUSE_DIR` and `RAW_DIR` are read by `pipelines/common/spark.py:5-6` to know where the data lives.



Figure 2. Where the 150 s of `docker compose build` went (bar widths proportional to time). Pip is the slowest layer; a second build reuses every cached layer and takes seconds.

Why it matters for PipeDoctor

Rebuild only when `Dockerfile` or `requirements.txt` change. When the agent later runs a candidate fix in its sandbox, it will use this same image, so "it works in the sandbox" means "it works here".

4 Step 2 — Download the data

> **Goal.** Get real data: three months (Jan–Mar 2024) of NYC yellow-taxi trips, about 9.5 million rows.

Command

```
docker compose run --rm lakehouse python scripts/download_taxi_data.py --start 2024-01 \
    --months 3
```

`compose run` starts a *new container* from the image, mounts your project as `/workspace` and `./data` as `/data` (`docker-compose.yml:7-9`), runs the command, and `--rm` deletes the container afterwards. The files stay on your PC in `data/raw/yellow/`.

Code behind it

```
29      for ym in month_range(a.start, a.months):
30          dest = os.path.join(a.out, f"yellow_tripdata_{ym}.parquet")
31          if os.path.exists(dest):
32              print(f"skip   {dest} (already there)")
33              continue
34          url = BASE.format(ym=ym)
35          print(f"get    {url}")
36          tmp = dest + ".part"
37          urllib.request.urlretrieve(url, tmp)
38          os.replace(tmp, dest)
39          print(f"saved  {dest} ({os.path.getsize(dest) / 1e6:.1f} MB)")
```

- `scripts/download_taxi_data.py` (line 10): `BASE` is the public CloudFront address of the TLC files, one file per month.
- Lines 31–33: **skip if present**, so running twice never downloads twice.
- Lines 36–38: download to `*.part`, then `os.replace` renames it. A file that exists under its real name is therefore always complete.

What your output showed

```
saved  /data/raw/yellow/yellow_tripdata_2024-01.parquet (50.0 MB)
saved  /data/raw/yellow/yellow_tripdata_2024-02.parquet (50.3 MB)
saved  /data/raw/yellow/yellow_tripdata_2024-03.parquet (60.1 MB)
```

160.4 MB in total. The line `Network pipedoctor_default Created` is Compose making a private network; we do not use it.

Why it matters for PipeDoctor

File-arrival problems come later (a *late* file is failure scenario 4, which PipeDoctor must escalate rather than patch). The `.part` rename is what guarantees that a visible file is a complete one.

5 Step 3 — Unit tests

> **Goal.** Prove the transformation logic is correct on tiny hand-made tables *before* spending minutes on 9.5 million rows.

Command

```
docker compose run --rm lakehouse python -m pytest pipelines/tests -q
```

`-q` means quiet: one dot per passing test. Your output, `........  8 passed in 34.78s`, is 8 dots = 8 tests. The 35 s is mostly Spark starting its Java engine; the repeat took 18 s.

Code behind it

```
 6 @pytest.fixture(scope="session")
 7 def spark():
 8     s = get_spark("tests", delta=False)  # transforms are tested without Delta
 9     yield s
10     s.stop()
```

`pipelines/tests/conftest.py` (lines 6–10): one Spark session shared by all tests, *without* Delta (`pipelines/common/spark.py:24` only loads Delta when `delta=True`). Possible because the transforms are pure DataFrame-in, DataFrame-out functions (`transforms.py:1`).

Line	Test (`test_transforms.py`)	Protects
36	`test_normalize_columns_lowercases`	`normalize_columns` `transforms.py:49`
42	`test_bronze_applies_contract_types`	`apply_bronze_contract` `:54`, `to_bronze` `:61`
50	`test_silver_renames_and_derives`	`to_silver_candidates` `:73`
58	`test_silver_missing_optional_column_is_null`	`_opt` `:68`
64	`test_silver_missing_required_column_fails_loudly`	direct column references, `:73–123`
72	`test_split_quarantines_each_invalid_rule`	`quarantine_reason` `:126`, `split_valid` `:140`
90	`test_duplicates_collapse_on_trip_id`	`dropDuplicates` `:146`
95	`test_gold_aggregates_per_day_and_zone`	`to_gold` `:152`

Why it matters for PipeDoctor

In the agent's *Validate* step, a candidate fix must keep these 8 tests green. The same `pytest` command becomes the CI job of the sprint plan's Step 3 (GitHub).

6 Step 4 — Bronze: land the raw files

> **Goal.** Copy each raw file into a Delta table *as received*, plus two audit columns, without cleaning anything yet.

Command (steps 4, 5 and 6 are this one command)

```
docker compose run --rm lakehouse python -m pipelines.run_all
```

```
 8 def main() -> None:
 9     spark = get_spark("pipedoctor-run-all")
10     t0 = time.time()
11     bronze.run(spark)
12     silver.run(spark)
13     gold.run(spark)
14     print(f"[run_all] green in {time.time() - t0:.0f}s")
```

`pipelines/run_all.py` (lines 8–14): start Spark once (`pipelines/common/spark.py:14–34`, local mode, Delta switched on), run the three layers in order, print the elapsed time. If any layer raises an exception, the last line is never printed: that is how a "red" run looks.

Code behind bronze

```
36          df = to_bronze(spark.read.parquet(path), name)
37          (
38              df.write.format("delta")
39              .mode("append")
40              .option("mergeSchema", "true")
41              .save(BRONZE_PATH)
42          )
```

```
61  def to_bronze(df: DataFrame, source_file: str) -> DataFrame:
62      df = apply_bronze_contract(normalize_columns(df))
63      return df.withColumn("_source_file", F.lit(source_file)).withColumn(
64          "_ingested_at", F.current_timestamp()
65      )
```

- `bronze.py:26` lists the `*.parquet` files in `data/raw/yellow`; `bronze.py:31–44` loops over them **one file at a time**.
- `transforms.py:49–51`: lower-case all column names (TLC mixes `Airport_fee` and `airport_fee`).
- `transforms.py:13–34 BRONZE_CONTRACT`: the agreed type of each column (for example `trip_distance` is `double`). `transforms.py:54–58` casts to it, so every month lands with the same types.
- `transforms.py:63–64`: add `_source_file` and `_ingested_at`. The first one is how the pipeline remembers what it already loaded.
- `bronze.py:40`: `mergeSchema=true` lets a *new* column be added to bronze. A changed column *type* is not absorbed and fails the append (see the docstring, `bronze.py:1–5`).

What your output showed

```
[bronze] ingested yellow_tripdata_2024-01.parquet
[bronze] ingested yellow_tripdata_2024-02.parquet
[bronze] ingested yellow_tripdata_2024-03.parquet
[bronze] done: 3 new file(s), 9,554,778 rows in bronze
```

The long `Ivy ... resolving dependencies` block just before is Spark locating the Delta jars; thanks to the warm-up in step 1 it needed no download.

Why it matters for PipeDoctor

This is the first place real failures appear. **Scenarios 1–2** (schema drift, type change) hit `BRONZE_CONTRACT` and `mergeSchema`.

7 Step 5 — Silver: clean, check, quarantine

> **Goal.** Turn bronze into trustworthy trips: typed, renamed, de-duplicated; anything invalid is set aside *with a reason*, never silently lost.

Code behind silver

```
29      bronze = spark.read.format("delta").load(BRONZE_PATH)
30      valid, quarantined = split_valid(to_silver_candidates(bronze))
31
32      if DeltaTable.isDeltaTable(spark, SILVER_PATH):
33          (
34              DeltaTable.forPath(spark, SILVER_PATH)
35              .alias("t")
36              .merge(valid.alias("s"), "t.trip_id = s.trip_id")
```

```
            .whenNotMatchedInsertAll()
            .execute()
        )
    else:
        valid.write.format("delta").partitionBy("pickup_date").save(SILVER_PATH)
    ensure_constraints(spark)

    quarantined.write.format("delta").mode("overwrite").option("overwriteSchema", "
    true").save(QUARANTINE_PATH)
```

- `silver.py:29–30`: read bronze, rename and derive columns (`transforms.py:73–123`), then split into `valid` and `quarantined` (`transforms.py:140–149`).
- **trip_id** (`transforms.py:107–123`): the taxi data has no ID, so we build one: a SHA-256 hash of vendor, pickup time, dropoff time, both zones, distance, fare and total. Same trip, same ID, every time. This is what makes de-duplication and MERGE possible.
- `silver.py:32–41`: first run, the table does not exist, so it is created and **partitioned by day** (`pickup_date`). Later runs use **MERGE**: insert only trips whose `trip_id` is not already there.
- `silver.py:42` and `silver.py:11–15`: Delta **CHECK constraints** (fare ≥ 0, distance ≥ 0, pickup $\leq$ dropoff) make the table itself refuse bad rows. They hold because bad rows are quarantined first.
- `silver.py:44`: the quarantine table is rewritten each run, with the extra column `quarantine_reason`.

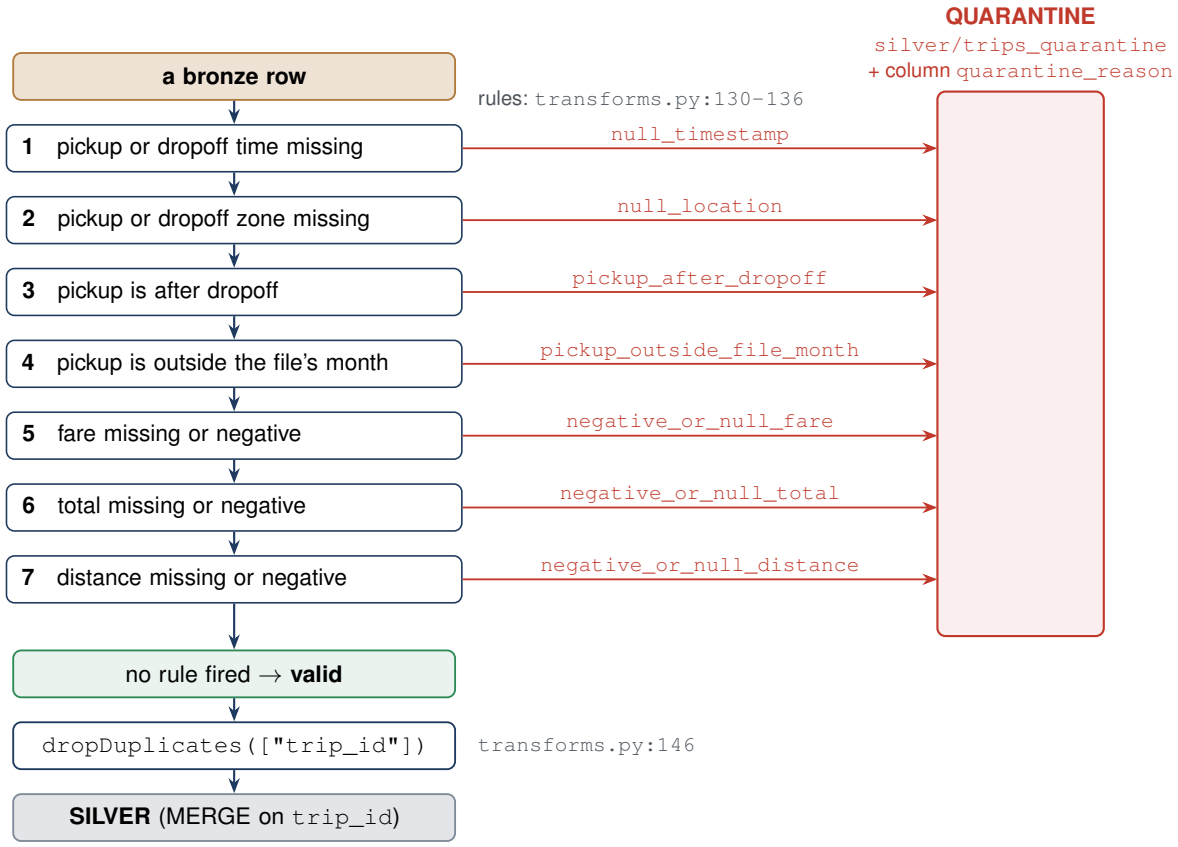


Figure 3. The silver filter (`quarantine_reason()`, `transforms.py:126–137`). Rules are checked top to bottom; **the first rule that fails names the reason**, and the row goes to quarantine.

What your output showed

```
[silver] done: 9,417,635 trips, 137,142 quarantined
```

About 1.4 % of rows (137 142 of 9 554 778) were set aside. That is plausible for raw taxi data (wrong dates, negative fares). We do not know the split per reason yet: see Section 12.

Why it matters for PipeDoctor

Silver is where most scenarios land: **3** invalid values (quarantine rules and CHECK constraints), **8** duplicates after a retry (MERGE on `trip_id`), **11** a column removed upstream (`to_silver_candidates` refers to required columns directly, so it fails loudly with an `AnalysisException`).

8 Step 6 — Gold: the business table

> **Goal.** Answer a business question quickly: *how many trips and how much revenue per pickup zone per day?*

Code behind gold

```
152 def to_gold(silver: DataFrame) -> DataFrame:
153     return silver.groupBy("pickup_date", "pu_location_id").agg(
154         F.count("*").alias("trips"),
155         F.round(F.sum("total_amount"), 2).alias("revenue"),
156         F.round(F.avg("trip_distance"), 3).alias("avg_distance"),
157         F.round(F.avg("tip_amount"), 3).alias("avg_tip"),
158     )
```

`gold.py:8-9` reads silver, applies `to_gold`, and **overwrites** the gold table. Gold is cheap to rebuild, so it is simply recomputed from silver every run, never merged.

What your output showed

```
[gold] done: 20,623 rows
[run_all] green in 567s
```

Sanity check: Jan–Mar 2024 is 91 days (2024 is a leap year), and 20 623 / 91 ≈ 227 zones with at least one trip per day. The city has about 265 zone IDs, so this is a sensible number. Each gold row summarises on average 9 417 635 / 20 623 ≈ 457 trips.

Why it matters for PipeDoctor

Gold is where a silver problem becomes visible to the business (missing days, odd totals). The agent's `get_lineage` tool will follow the arrow back: gold ← silver ← bronze ← raw file.

9 Step 7 — Run it again (idempotence)

> **Goal.** Prove that re-running, which is exactly what happens on a retry, creates no duplicates and changes no numbers.

Command

The same `python -m pipelines.run_all` as step 4–6.

Code that makes it safe

```
29     done = already_ingested(spark)
30     new = 0
31     for path in files:
32         name = os.path.basename(path)
33         if name in done:
34             print(f"[bronze] skip {name} (already ingested)")
35             continue
```

`bronze.py:29-35`: bronze skips any file whose name is already in `_source_file` (`bronze.py:17-21`). Silver uses `whenNotMatchedInsertAll` (`silver.py:36-38`): trips that already exist are

left alone. Quarantine and gold are *rebuilt*, which is also safe.

Layer	Run 1 (first time)	Run 2 (re-run)
Bronze	ingested 3 files, 9 554 778 rows	`skip` × 3, 0 new, still 9 554 778
Silver	table created, 9 417 635 trips	MERGE inserted 0, still 9 417 635
Quarantine	rewritten, 137 142 rows	rewritten, 137 142 rows
Gold	20 623 rows	recomputed, 20 623 rows
Time	567 s	134 s

The ≈ 433 s difference is roughly the cost of writing the 9.5 million bronze rows (our estimate, not measured separately). The remaining 134 s is reading, checking and counting.

> **Why this is the most important step for PipeDoctor.** When a job fails, the standard human reaction is "just re-run it". That is only safe if the pipeline is idempotent. Every fix PipeDoctor proposes must keep this property, and scenario 8 (duplicates after a retry) is the test of it.

10 Reading the numbers

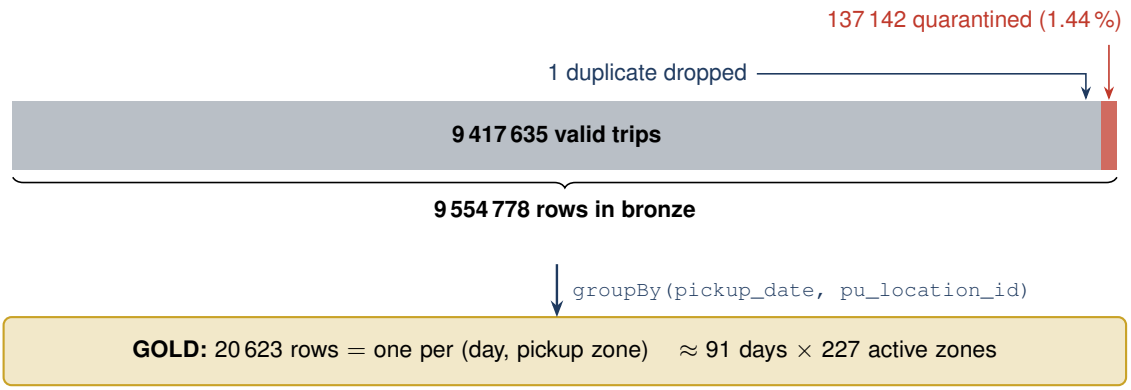


Figure 4. Row accounting from raw to gold. The quarantine slice is drawn to scale; the duplicate is too small to see.

Quantity	Rows	Where it is counted
Bronze	9 554 778	`bronze.py:45`
− Quarantined	137 142	`silver.py:48`
− Duplicates dropped on `trip_id`	1	deduced, `transforms.py:146`
= Silver	9 417 635	`silver.py:47`

The "1". The log never prints it, but the logic forces it. Every bronze row goes either to quarantine or to the valid set (`transforms.py:143–148`), and the only step that can remove a valid row is `dropDuplicates`. Since $9\,554\,778 - 137\,142 - 9\,417\,635 = 1$, exactly one row shared its `trip_id` with another. The ID hashes eight fields, so these two rows were identical in all eight.

11 Noise versus signal in the log

Much of the output is harmless chatter. Knowing what to ignore is a real skill for a data engineer, and the first thing PipeDoctor has to learn.

Message	Where	Why it is harmless
`debconf: unable to initialize frontend`	step 1 (`#8`)	The build has no interactive terminal; apt falls back to non-interactive mode.
`No JRE found. Skipping Java certificates`	step 1 (`#8`)	Order of installation inside apt; the certificates are added a few lines later (`Adding debian:...`).
`Running pip as the 'root' user`	step 1 (`#10`)	We are inside a throw-away container; nothing on your PC is affected.
`hostname resolves to a loopback address`	step 1 (`#11`)	Spark picks the container's own IP for local mode.
`Unable to load native-hadoop library`	steps 1 and 4–7	Spark falls back to pure-Java code. Normal in Docker.
`Truncated the string representation of a plan`	steps 4–7	Spark only shortens a debug print of a long query plan.
`0 artifacts copied, 3 already retrieved`	steps 4–7	**Good news:** the Delta jars come from the image (warm-up, step 1).

Signal to look for instead: a line starting with `[bronze]`, `[silver]`, `[gold]` or `[run_all]`; or a Java/Python stack trace ending in an `Exception`. Those lines are what PipeDoctor will read through `get_run_logs` and `get_spark_error`.

12 What the output does *not* prove yet, and what to do next

1. **Why were 137 142 rows quarantined?** The log gives the total, not the reasons. This one command prints the split per rule (we have *not* run it, so treat the result as new information):

```
docker compose run --rm lakehouse python -c "from pipelines.common.spark import *; s\
    =get_spark(); s.read.format('delta').load(QUARANTINE_PATH).groupBy('\
    quarantine_reason').count().show()"
```

2. **Move the project out of OneDrive.** Your folder is `C:\Users\Ayman\OneDrive\Desktop\...`, and `data/` now holds the raw files and three Delta tables (likely around a gigabyte). OneDrive will try to sync all of it, which is slow and can lock Delta files. Copy the project to `C:\dev\pipedoctor` (the sprint plan's Step 3, Git, needs this too). `data/` is already in `.gitignore`.

3. **Look inside the lakehouse.** Open `data/lakehouse/bronze/yellow_trips/_delta_log/`: each JSON file is one commit (one per ingested file). This is the *Delta history* that PipeDoctor reads with `describe_delta_history`.

4. **Next sprint step:** the sprint plan's Step 3, a GitHub repository and CI that runs the same `pytest` command on every push.

13 How each step connects to PipeDoctor

#	Step	What PipeDoctor will do with it
1	**Image**	Run candidate fixes in the *same* environment (`run_sandbox_job`).
2	**Download**	Scenario 4: a late file is escalated to a human, not patched.
3	**Tests**	The *Validate* step: a fix that breaks one of the 8 tests is rejected.
4	**Bronze**	Scenarios 1–2: schema drift and type change (`BRONZE_CONTRACT`, `mergeSchema`).
5	**Silver**	Scenarios 3, 8, 11: invalid values, duplicates after a retry, column removed upstream.
6	**Gold**	Where the business notices; `get_lineage` walks back to the cause.
7	**Re-run**	Every fix must preserve idempotence, otherwise a retry creates duplicates.

14 Cheat sheet: the five commands

```
# step 1  (only again if Dockerfile or requirements.txt change)
docker compose build

# step 2  download the data
docker compose run --rm lakehouse python scripts/download_taxi_data.py --start 2024-01 \
    --months 3

# step 3  unit tests
docker compose run --rm lakehouse python -m pytest pipelines/tests -q

# steps 4-6  bronze, silver, gold   (run it twice = step 7)
docker compose run --rm lakehouse python -m pipelines.run_all

# bonus  open a shell inside the container
docker compose run --rm lakehouse bash
```

Rule of thumb: **code changes need no rebuild** (the project folder is mounted), **Dockerfile or requirements changes do**.